

Rapport MasterCamp

Membres de l'équipe (groupe 14) :

- Mael Lecarpentier
- Anaïs Robert
- Florian Leroux
- Edgar Tromparent
- Adam Benadjaoud
- Theo Ung

Sommaire :

Liste et répartition des tâches :	2
Tâche 1 : LED du HAL3.1 et LED RGB des 2 feux avant (test_LED.py) :	3
Tâche 2 : LED WS2812 LED (test_LED2.py) :	4
Tâche 3 : Servomoteurs (test_servo.py) :	5
Tâche 4 : Moteur DC (test_motor.py) :	6
Tâche 5 : Capteur de mesure ultrason (test_ultra.py) :	6
Tâche 6 : Capteur de suivi de ligne (test_line_tracking.py)	7
Tâche 7 : Intégration des fonctions (test_suivi_ligne.py , test_obstacles.py)	8
Tâche 8 : Capteur de suivi de lumière (test_light_tracking.py)	9

Liste et répartition des tâches :

N°	Nom de la tâche	Objectif	Date de début	Date de fin
0	Montage du robot	Avoir un robot opérationnel	06/06/2025	06/06/2025
0 bis	Mise en route du Raspberry	Finir la configuration du robot	10/06/2025	10/06/2025
1	LED de la HAL3.1 et LED RGB des feux avant	Piloter les 9 LED (HAL3.1 + feux avant)	10/06/2025	10/06/2025
2	LED WS2812	Piloter les LED WS2812	10/06/2025	11/06/2025
3	Servomoteurs	Piloter les 3 servomoteurs	11/06/2025	12/06/2025
4	Moteur DC	Piloter le moteur DC	11/06/2025	11/06/2025
5	Capteur ultrason	Mesurer la distance à un obstacle	11/06/2025	11/06/2025
6	Capteur de suivi de ligne	Mesurer l'état des 3 capteurs infrarouges	11/06/2025	12/06/2025
7.1	Intégration : suivi de ligne	Créer le module de suivi de ligne	12/06/2025	
7.2	Intégration : évitement d'obstacle	Créer le module d'évitement d'obstacle	12/06/2025	12/06/2025
7.3	Assemblage des fonctions 7.1 et 7.2	Assembler les fonctions précédentes		
8	Capteur de suivi de lumière	Mesurer l'état des capteurs de lumière	12/06/2025	12/06/2025

N° de la tâche	Mael	Anais	Florian	Edgar	Adam	Theo
0	✓	✓	✓	✓	✓	✓
0 bis	✓					✓
1	✓	✓				✓
2			✓	✓	✓	
3			✓	✓		
4					✓	✓
5	✓	✓				
6	✓	✓				
7.1	✓	✓				✓
7.2			✓	✓	✓	
7.3						
8						✓

Tâche 1 : LED du HAL3.1 et LED RGB des 2 feux avant (test_LED.py) :

Explication du testbench :

Nous avons mis en place un testbench nous permettant d'allumer une des trois LED de la carte HAT 3.1 ou les 3x2 LED RGB à l'avant du robot en fonction de la saisie utilisateur.

Voici le fonctionnement :

- Le premier caractère définit l'action : « 0 » pour éteindre et « 1 » pour allumer
- Le deuxième caractère définit la LED cible

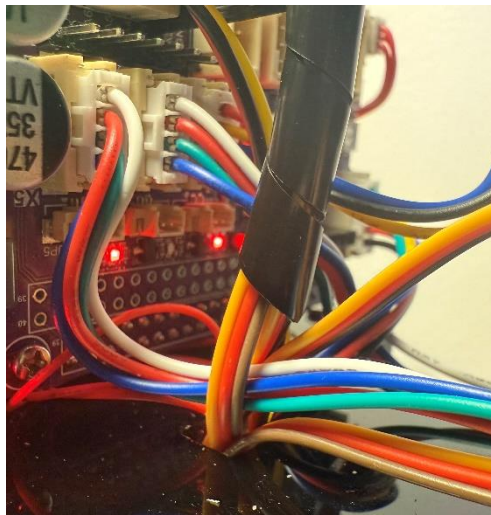
Ex : « 11 » pour allumer la LED(1), « 03 » pour éteindre la LED(3), ...

Pour arrêter le programme proprement, il suffit de saisir : « exit »

Voici une capture du terminal après exécution :

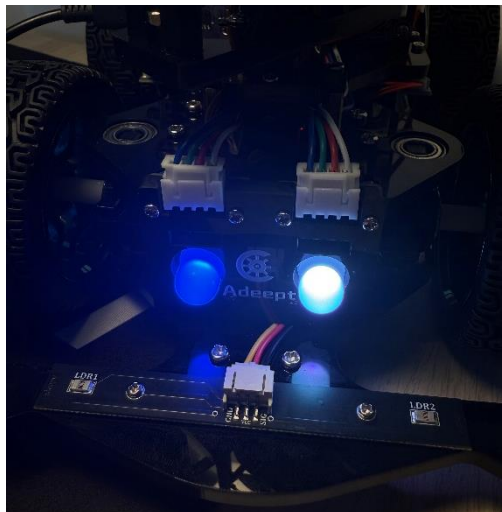
```
File Edit Tabs Help
Veuillez entrer votre choix :11
LED(9) ON
Veuillez entrer votre choix :12
LED(25) ON
Veuillez entrer votre choix :02
LED(25) OFF
Veuillez entrer votre choix :13
LED(11) ON
Veuillez entrer votre choix :exit
Fin du programme

-----
(program exited with code: 0)
Press return to continue
```



```
File Edit Tabs Help
Veuillez entrer votre choix :14
LED(0) ON
Veuillez entrer votre choix :15
LED(19) ON
Veuillez entrer votre choix :16
LED(13) ON
Veuillez entrer votre choix :19
LED(6) ON
Veuillez entrer votre choix :exit
Fin du programme

-----
(program exited with code: 0)
Press return to continue
```



Remarque : Lors de l'exécution du code, nous nous sommes rendu compte que le comportement des LEDs RGB était inverse à celui attendu (les LEDs s'allument lors de l'exécution de LED(X).off et inversement)

Tâche 2 : LED WS2812 LED (test_LED2.py) :

Explication du testbench :

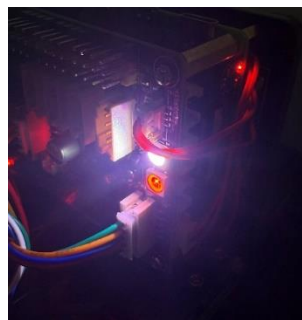
Nous avons mis en place un testbench nous permettant d'allumer la LED de notre choix avec une saisie utilisateur comprenant les paramètres suivant :

- Numéro de la LED (de 0 à 13)
- Couleur de la LED (red (R), green (G), blue (B), white (W), black (N))
- Luminosité de la LED (de 0 à 255)

Nous avons également ajouté la possibilité d'allumer toutes les LED une par une avec la saisie : « all ». Les LED s'allument en blanc en partant de la LED(0) allant jusqu'à la LED(13) puis s'éteignent. Lorsque l'utilisateur saisie : « exit », le programme s'arrête.

Voici une capture du terminal après exécution :

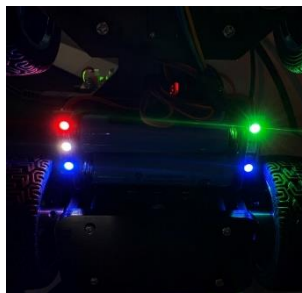
```
File Edit Tabs Help
Selection sous la forme : (num_LED, couleur, luminosité)
0,R,25
Selection sous la forme : (num_LED, couleur, luminosité)
1,W,255
Selection sous la forme : (num_LED, couleur, luminosité)
exit
-----
(program exited with code: 0)
Press return to continue
```



```
File Edit Tabs Help
Selection sous la forme : (num_LED, couleur, luminosité)
2,B,255
Selection sous la forme : (num_LED, couleur, luminosité)
3,W,125
Selection sous la forme : (num_LED, couleur, luminosité)
4,G,25
Selection sous la forme : (num_LED, couleur, luminosité)
5,R,50
Selection sous la forme : (num_LED, couleur, luminosité)
6,B,25
Selection sous la forme : (num_LED, couleur, luminosité)
7,G,100
Selection sous la forme : (num_LED, couleur, luminosité)
8,N,255
Selection sous la forme : (num_LED, couleur, luminosité)
exit
-----
(program exited with code: 0)
Press return to continue
```



```
File Edit Tabs Help
Selection sous la forme : (num_LED, couleur, luminosité)
8,B,125
Selection sous la forme : (num_LED, couleur, luminosité)
9,W,25
Selection sous la forme : (num_LED, couleur, luminosité)
10,R,255
Selection sous la forme : (num_LED, couleur, luminosité)
11,G,25
Selection sous la forme : (num_LED, couleur, luminosité)
12,N,255
Selection sous la forme : (num_LED, couleur, luminosité)
13,B,75
Selection sous la forme : (num_LED, couleur, luminosité)
exit
-----
(program exited with code: 0)
Press return to continue
```



Remarque : On utilise la déclaration fourni dans le programme de test du robot pour le moment. Lors de notre code final, la déclaration du bandeau de LED sera entièrement retravaillée.

Tâche 3 : Servomoteurs (test_servo.py) :

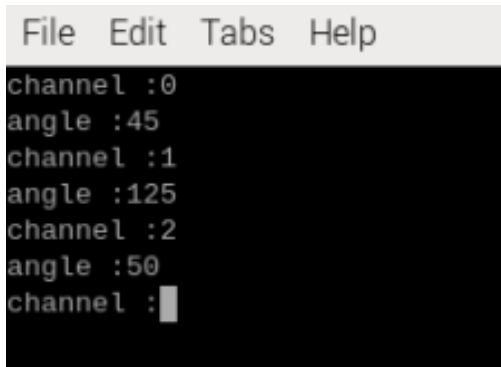
Nous avons mis en place un testbench qui nous permet de faire tourner indépendamment chaque servo moteur à l'angle souhaité grâce à la saisie utilisateur.

Channel 0 : servo de contrôle de la direction des roues avant

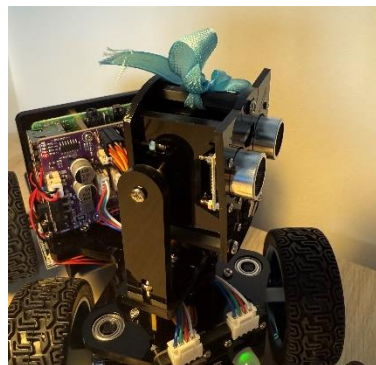
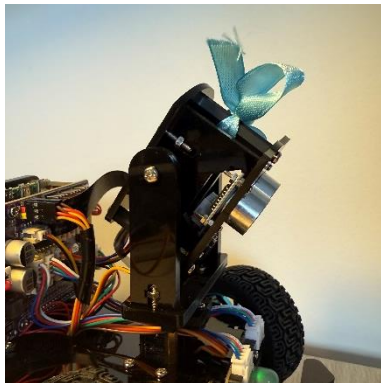
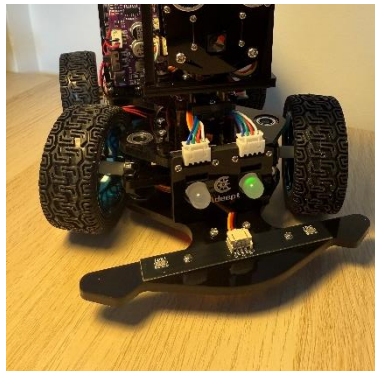
Channel 1 : servo de contrôle de la rotation de la tête (module ultrason)

Channel 2 : servo de contrôle de l'inclinaison de la tête (module ultrason)

Voici une capture du terminal après exécution :



```
File Edit Tabs Help
channel :0
angle :45
channel :1
angle :125
channel :2
angle :50
channel : 
```



Remarque : Après exécution du programme de test, nous avons défini des plages de rotations pour chaque servo et leurs position neutre :

Channel 0 : de 40° à 140°, position neutre à 90°

Channel 1 : de 20° à 160°, position neutre à 90°

Channel 2 : de 45° à 125°, position neutre à 85°

Tâche 4 : Moteur DC (test_motor.py) :

Nous avons mis en place un testbench qui déplace le robot en 4 phases :

- Phase 1 : Le robot avance
- Phase 2 : Le robot ralenti progressivement jusqu'à l'arrêt (tout en avançant)
- Phase 3 : Le robot recule
- Phase 4 : Le robot ralenti progressivement jusqu'à l'arrêt (tout en reculant)

Une fois cette boucle fini, le programme s'arrête.

Voici une capture du terminal après exécution :

```
File Edit Tabs Help
Forward
Slow down
Backward
Slow down
Fin du programme

-----
(program exited with code: 0)
Press return to continue
```

Tâche 5 : Capteur de mesure ultrason (test_ultra.py) :

Explication du testbench :

Nous avons mis en place un testbench qui nous permet de mesurer la distance entre le robot et un obstacle placé devant lui. La distance de détection maximale est de 2m selon la documentation technique du capteur, pour notre test, nous avons choisi une distance maximale de 50cm. Nous obtenons une distance en « mm » dans le terminal. Lorsque le programme a effectué 5 mesure (chacune espacées de 2 secondes) le programme s'arrête.

Voici une capture du terminal après exécution :

```
File Edit Tabs Help
/usr/lib/python3/dist-packages/gpiozero/input_devices.py:852: PWMSoftwareFallback
k: For more accurate readings, use the pigpio pin factory. See https://gpiozero.r
eadthedocs.io/en/stable/api_input.html#distancesensor-hc-sr04 for more info
  warnings.warn(PWMSoftwareFallback(
Aucune detection
Objet detecte a 356 mm
Objet detecte a 195 mm
Objet detecte a 75 mm
Objet detecte a 29 mm
Fin du programme

-----
(program exited with code: 0)
Press return to continue
```


Tâche 6 : Capteur de suivi de ligne (test_line_tracking.py)

Explication du testbench :

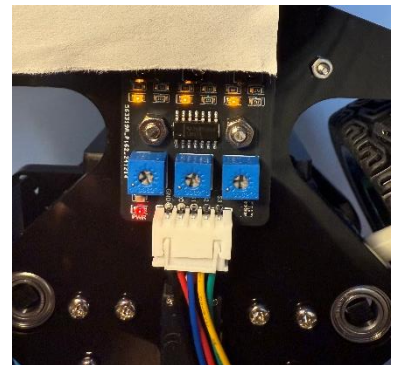
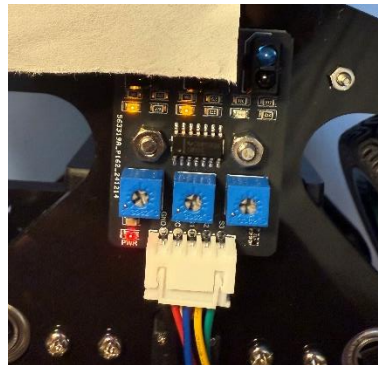
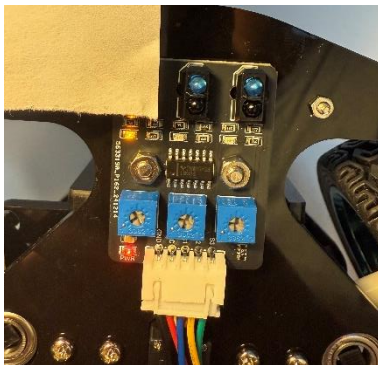
Nous avons mis en place un testbench nous permettant de récupérer les données lu par le capteur infrarouge. Nous observons que : 0 correspond à la détection d'une surface noir (absorbante) et la valeur 1 correspond à la détection d'une surface clair (réfléchissante). Nous traitons ensuite le signal obtenu et interprétons le résultat de la manière suivante :

- 3 LED à 0 : Le robot est hors-circuit, il doit reculer pour retourner sur la ligne
- LED gauche à 0 : Le robot doit tourner à droite pour se recentrer
- LED droite à 0 : Le robot doit tourner à gauche pour se recentrer
- 3 LED à 1 : Le robot peut avancer tout droit

Voici une capture du terminal après exécution :

```
File Edit Tabs Help
ACTION: Avancer tout droit
left: 0 middle: 1 right: 0
ACTION: Tourner a droite
left: 0 middle: 1 right: 1
ACTION: Tourner a gauche
left: 1 middle: 1 right: 0
ACTION: Reculer
left: 0 middle: 0 right: 0

-----
(program exited with code: 0)
Press return to continue
```

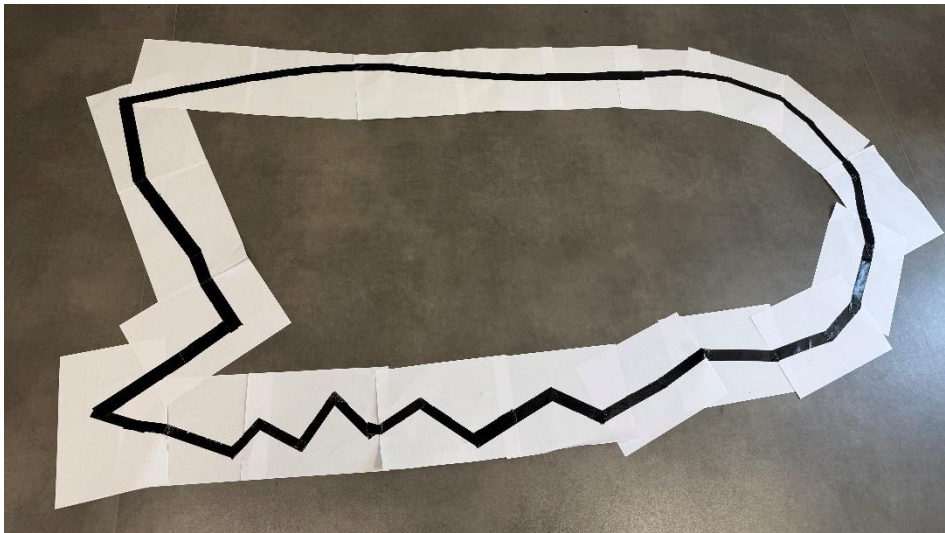


Tâche 7 : Intégration des fonctions (test_suivi_ligne.py , test_obstacles.py)

Nous avons commencé à intégrer les fonctions unitaire de chaque capteur et moteur pour faire un programme complet. Nous nous sommes dans un premier temps intéressé au suivi de ligne et à l'évitement d'obstacle.

Suivi de ligne :

Nous avons construit un circuit pour tester notre robot et ajuster notre programme. Le robot est capable de suivre une ligne légèrement ondulée. Pour ce qui est des virages, l'angle de braquage des roues reste très limitant mais le robot semble suivre une trajectoire acceptable.



Remarque : Lors de test unitaire plus poussé sur les servo-moteur, nous observons une latence non négligeable lors de l'envoi et l'appel répétitif des fonctions de rotations. Même après avoir augmenté la fréquence du PCA9685 à 200Hz (initialement à 50Hz), les résultats restent approximatif lors de la recherche de réactivité. Nous envisageons de repasser sur un pin PWM natif de la carte Raspberry et utiliser la librairie *pigpio*.

Evitement d'obstacle :

Le robot est capable d'éviter un carton de petite taille, il détecte l'objet placer devant lui et amorce la phase d'évitement par la suite. Pour le moment, n'ayant pas encore combiné évitement et suivi de ligne, l'évitement se fait seulement avec des délais.

Nous sommes en train de mettre au point un scanner avec le capteur ultrason. Il nous permettra de déterminer si l'évitement le plus simple doit se faire par la droite ou par la gauche de l'obstacle. Avec ce scan, nous prévoyons d'ajouter une représentation en temps réels des données reçu avec le capteur.

Tâche 8 : Capteur de suivi de lumière (test_light_tracking.py)

Nous avons mis en place un testbench capable de mesurer l'état des 2 capteurs de lumières. Une fois la mesure effectuée, le robot tourne en direction du capteur avec la valeur la plus élevée.

```
File Edit Tabs Help
Calibration
Calibration terminee. Seuil de tolerance = 4146
LDR Gauche = 64768, LDR Droite = 64768
Robot va tout droit
LDR Gauche = 60928, LDR Droite = 51200
  Robot tourne a gauche
LDR Gauche = 48128, LDR Droite = 65280
Robot tourne a droite
```

Remarque : A part lors de l'initialisation, il est difficile d'obtenir 2 valeurs parfaitement égal. Nous envisageons d'ajouter un delta dans nos calcul, nous permettant d'avoir une mesure plus réaliste de l'environnement et par la même occasion permettre au robot d'avancer tout droit.